

NutriApp — Roadmap de negocio

Estrategia de monetización, modelo freemium y plan de adquisición de usuarios.

Índice

- 1. [Modelo freemium](#)
- 2. [Roadmap técnico para implementarlo](#)
- 3. [Prompt para implementar el modelo en el código](#)
- 4. [Estrategia de adquisición de usuarios](#)
- 5. [Métricas para saber si va bien](#)
- 6. [Web vs App móvil — qué construir y cuándo](#)

1. Modelo freemium

Definición del plan gratuito (Free)

Lo que puede hacer cualquier usuario sin pagar:

Funcionalidad	Límite
Perfiles	1 perfil
Comidas registradas por día	Ilimitado
Recetas guardadas	Hasta 10
Consultas al AI (NutriChat)	10 por día
Resumen semanal básico	✓
Recetas de la comunidad	Solo lectura

Definición del plan Pro

Lo que se desbloquea pagando (~\$5 USD/mes):

Funcionalidad	Pro
Perfiles	Ilimitados (ideal para familias)
Recetas guardadas	Ilimitadas
Consultas al AI	Ilimitadas
Análisis mensual con gráficos	✓
Exportar resumen en PDF	✓
Crear recetas públicas en la comunidad	✓
Soporte prioritario	✓

Precio sugerido

- **Free:** gratis para siempre
- **Pro mensual:** \$4.99 USD/mes
- **Pro anual:** \$39.99 USD/año (ahorro del 33%)

Por qué este modelo funciona

El plan gratuito es lo suficientemente útil para que el usuario se enganche y construya el hábito de registrar lo que come. Los límites (especialmente el AI limitado a 10/día) crean fricción real sin bloquear la funcionalidad core. El usuario que usa el AI activamente y llega al límite tiene alta intención de pago.

2. Roadmap técnico para implementarlo

Estos son los cambios necesarios en el código, en orden de prioridad:

Paso 1 — Agregar campo `plan` al usuario

En el schema de Prisma, agregar a `User` :

```
plan      String   @default("free") // "free" | "pro"
planExpiry DateTime?           // null = gratis para siempre
```

Migración: `npx prisma migrate dev --name add_user_plan`

Paso 2 — Crear sistema de permisos en el backend

Un middleware `checkPlan` que:

- Lee el `plan` del usuario desde la DB
- Verifica si `planExpiry` está vigente
- Adjunta `req.userPlan` para que las rutas lo usen

Paso 3 — Aplicar límites por endpoint

- `POST /profiles` : rechazar si `plan === "free"` y ya tiene 1 perfil
- `POST /recipes` : rechazar si `plan === "free"` y ya tiene 10 recetas
- Contador de AI queries: tabla nueva `AiUsage { userId, date, count }` — rechazar si `count >= 10` y `plan === "free"`

Paso 4 — Integrar pasarela de pagos

Opciones recomendadas (gratuito para integrar, cobran % por transacción):

- **Stripe** — mejor documentación, funciona con tarjetas internacionales
- **MercadoPago** — mejor para Argentina/LATAM, muy usado localmente

El flujo sería:

1. Usuario hace click en "Ir a Pro"
2. Frontend redirige al checkout de Stripe/MercadoPago
3. Webhook de Stripe notifica al backend cuando el pago se confirma
4. Backend actualiza `plan = "pro"` y `planExpiry = now + 30 días`

Paso 5 — UI de upgrade

- Banner no intrusivo cuando el usuario llega al límite de AI
- Página `/pricing` con comparación de planes
- Badge "Pro" visible en el perfil

3. Prompt para implementar el modelo en el código

Cuando estés listo para implementar el modelo freemium, usá este prompt exacto con Claude:

PROMPT PARA COPIAR Y PEGAR:

Quiero implementar un modelo freemium en NutriApp. El proyecto es un monorepo con:

- Frontend: Next.js (App Router) en apps/web
- Backend: Express.js en apps/api/src/index.js
- ORM: Prisma con MySQL (schema en apps/api/prisma/schema.prisma)
- Auth: JWT en cookie HttpOnly, middleware en apps/api/src/middlewares/auth.js

Los planes son:

- Free: 1 perfil, máximo 10 recetas guardadas, máximo 10 consultas al AI por día
- Pro (\$4.99/mes): perfiles ilimitados, recetas ilimitadas, AI ilimitado

Lo que necesito que implementes, en este orden:

1. SCHEMA: Agregar a User los campos ``plan` String @default("free")`` y ``planExpiry` DateTime?``. Crear la migración de Prisma.

2. MIDDLEWARE DE PLAN: Crear `apps/api/src/middlewares/checkPlan.js` que lea el plan del usuario desde la DB, verifique si `planExpiry` está vigente (si existe), y adjunte ``req.userPlan`` ("free" | "pro") al request.

3. LÍMITE DE PERFILES: En POST `/profiles`, si `req.userPlan === "free"` y el usuario ya tiene 1 perfil, devolver 403 con mensaje claro indicando que el plan Pro permite perfiles ilimitados.

4. LÍMITE DE RECETAS: En POST `/recipes`, si `req.userPlan === "free"`, contar las recetas del perfil activo. Si ya tiene 10, devolver 403 con mensaje de upgrade.

5. LÍMITE DE AI: Crear tabla `AiUsage` { id, userId, date (solo fecha sin hora), count } en Prisma. En `apps/web/app/api/chat/route.ts`, antes de llamar a Gemini, consultar `AiUsage` del día actual para ese `userId`. Si `count >= 10` y `plan === "free"`, devolver 403 con `{ error: "limite_ai", reply: "Llegaste al límite de 10 consultas diarias del plan gratuito. Actualizate a Pro para consultas ilimitadas." }`. Si sigue, incrementar el contador (upsert).

6. WEBHOOK DE STRIPE: Crear endpoint POST `/webhooks/stripe` en el backend que reciba eventos de Stripe. Cuando el evento sea `"checkout.session.completed"`, actualizar `plan = "pro"` y `planExpiry = now + 30 días` para el `userId` correspondiente. El `userId` debe pasarse como metadata en el checkout session.

7. ENDPOINT DE CHECKOUT: Crear POST `/billing/create-checkout` en el backend (auth requerido) que use el SDK de Stripe para crear una checkout session con el precio mensual configurado en env `STRIPE_PRICE_ID`, pasando `userId` en metadata. Devolver la URL de checkout al frontend.

8. FRONTEND - LÍMITES: En `chat/page.tsx`, si la respuesta del AI devuelve status 403 con error

"limite_ai", mostrar un banner debajo del input (no un error genérico) con texto "Llegaste al límite diario de NutriChat gratis" y un botón "Ir a Pro" que redirija a /pricing.

9. FRONTEND - PRICING PAGE: Crear apps/web/app/(app)/pricing/page.tsx con una tabla comparativa de planes Free vs Pro. El botón de Pro hace POST a /api/proxy/billing/create-checkout y redirige a la URL de checkout de Stripe.

Variables de entorno necesarias a agregar:

- Backend: STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, STRIPE_PRICE_ID
- Frontend: NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY

Antes de empezar, leé los archivos actuales relevantes para entender el código existente y no romper nada. Implementá los cambios en el orden indicado, uno por uno, verificando que cada uno compile antes de continuar.

4. Estrategia de adquisición de usuarios

Esta estrategia está diseñada para conseguir los primeros 100-500 usuarios reales sin gastar dinero en publicidad.

Fase 1 — Primeros 50 usuarios (semanas 1-4)

Objetivo: validar que la app funciona con usuarios reales y conseguir feedback.

Acción 1: Red cercana (0 esfuerzo, alto impacto)

- Contarle a 20-30 personas cercanas (familia, amigos, conocidos del gym, compañeros).
- No pedirles que "la prueben" — pedirles que la **usen de verdad durante una semana** y te cuenten qué les molestó o faltó.
- Resultado esperado: 10-15 usuarios activos + feedback real para mejorar.

Acción 2: Grupos de WhatsApp y Telegram fitness

- Buscar grupos activos de: entrenamiento en casa, crossfit, running, nutrición, pérdida de peso.
- Publicar con este texto (adaptalo a tu voz):

"Estuve desarrollando una app para registrar comidas y tiene un asistente de IA que te explica tus macros y te da consejos según tu objetivo. Está en beta, es gratis. ¿A alguien le interesa probarla y darme feedback? Los primeros usuarios me ayudan a mejorarla."

- No publicar como publicidad. Publicarlo como alguien compartiendo algo que hizo.
- Resultado esperado: 20-30 usuarios nuevos por grupo activo.

Acción 3: Reddit

- Subreddits en español: r/argentina, r/es, r/fitness_es, r/nutricion
- Subreddits en inglés (más grandes): r/loseit, r/nutrition, r/1200isplenty, r/gainit, r/MacroFactor
- Publicar mostrando el AI en acción (captura del chat respondiendo una pregunta real de nutrición).
- El título importa: "Hice una app con IA que analiza tus comidas y te da consejos según tu objetivo — gratis, busco feedback"
- No spamear — un post por subreddit relevante, con semanas de diferencia.

Fase 2 — Crecer a 200-500 usuarios (mes 2-3)

Objetivo: tracción orgánica sin pagar ads.

Acción 4: Contenido en TikTok/Instagram Reels

- Mostrar casos de uso reales: grabarte usando el AI, preguntarle qué cenar según lo que comiste, ver el resumen semanal.
- Formato que funciona: pantalla del celular con la app, voz en off explicando lo que hace.
- Ideas de videos:
 - "Le pregunté a la IA de mi app de nutrición qué cenar y esto me dijo"
 - "Así registro mis comidas con IA (sin contar calorías a mano)"
 - "Mi app de nutrición me dijo que no llego a la proteína diaria"
- No necesitás edición fancy — el contenido del AI respondiendo es el gancho.
- Publicar 3-4 videos por semana durante 4 semanas.
- Resultado esperado: 1 de cada 10-15 videos va a tener tracción orgánica.

Acción 5: Product Hunt

- Lanzar en Product Hunt un martes o miércoles a las 00:01 AM Pacific Time.
- Preparar: descripción clara, capturas reales, GIF del AI respondiendo.
- Pedir votos a contactos el día del lanzamiento (es clave tener votos tempranos).
- Resultado esperado: 50-200 usuarios nuevos el día del lanzamiento si entrás en el top 10.

Acción 6: Comunidad de nutricionistas

- Contactar 10-15 nutricionistas en Instagram o LinkedIn.
- Ofrecerles cuenta Pro gratis a cambio de que lo recomienden a sus pacientes.
- Un nutricionista con 500-1000 seguidores que lo recomienda vale más que cualquier ad.

Fase 3 — Monetización (mes 3 en adelante)

Una vez que tenés 200+ usuarios activos:

1. Activar el plan Pro (\$4.99/mes).
2. Enviar email/notificación a usuarios que llegaron al límite de AI más de 3 veces.
3. Ofrecer 30 días de Pro gratis a los primeros 50 usuarios como recompensa por el feedback dado.
4. Meta realista: 5% de conversión free → Pro = 10 usuarios pagos de 200 = \$50/mes. Suficiente para cubrir los costos de infraestructura.

Qué NO hacer

- No pagar ads en Facebook/Instagram hasta tener al menos 200 usuarios orgánicos. Sin datos de conversión, tirar plata en ads es desperdicio.
- No hacer spam en grupos. Una sola publicación bien hecha vale más que 10 spams.
- No lanzar en Product Hunt sin preparación — es una sola bala.
- No construir más features sin validar con usuarios reales primero.

5. Métricas para saber si va bien

Estas son las métricas que importan, en orden:

Métrica	Qué mide	Meta mes 1	Meta mes 3
Usuarios registrados	Tracción	50	300
DAU (usuarios activos por día)	Retención	10	60

Consultas AI por usuario/semana	Engagement	5	10
Usuarios que llegaron al límite de AI	Intención de pago	—	20% de activos
Conversión free → Pro	Monetización	—	3-5%

Si el DAU es menor al 20% de los registrados, hay un problema de retención — los usuarios se registran pero no vuelven. Ahí hay que revisar la propuesta de valor o el onboarding.

6. Web vs App móvil — qué construir y cuándo

Esta es una de las decisiones más importantes del producto. La respuesta depende del momento, los recursos y los objetivos.

Situación actual

NutriApp es una web app en Next.js (App Router). Funciona en el navegador del celular, pero no es una app nativa. Ya está construida, probada y lista para deploy. Construir una app móvil nativa hoy sería empezar desde cero.

Comparación directa

Factor	Web App (actual)	App móvil nativa
Tiempo para lanzar	Hoy	3-6 meses extra
Costo de desarrollo	\$0 (ya está hecha)	Alto (React Native o Flutter)
Deploy y actualizaciones	Instantáneo (Vercel)	Aprobación de App Store (2-7 días)
SEO y descubrimiento orgánico	✅ Google puede indexarla	❌ No aplica
Compartir en redes sociales	✅ URLs directas	Requiere deep links
Push notifications	❌ No nativo	✅ Nativas del SO
Cámara / sensores del celu	Limitado	✅ Acceso completo
Experiencia UX en mobile	Buena si es responsive	Excelente
Distribución	Link directo, sin tiendas	App Store + Google Play
Monetización in-app	Stripe directo	Apple cobra 30%
Offline	❌ Requiere conexión	✅ Con cache nativo
Retención (ícono en home)	❌ Poca presencia	✅ Alta (vive en el launcher)

Punto crítico: Apple cobra el 30%

Si monetizás desde una app iOS con compras in-app, Apple se queda con el 30% de cada pago. En un plan de \$4.99/mes, te quedan \$3.49. Con una web app + Stripe, te quedás con ~\$4.63 (solo comisión de Stripe, ~2.9% + \$0.30). **La diferencia es estructural.**

Decisión recomendada: Web → PWA → Native

No es "web o mobile" — es una progresión natural:

Fase 1 — Lanzar la web app (Ahora)

- Deploy en Vercel (frontend) + Railway (backend + DB).
- La web app ya funciona en mobile desde el browser.
- Adquirir los primeros 50-200 usuarios, validar retención, iterar.
- Costo: \$0.

Fase 2 — Convertir a PWA (1-2 semanas de trabajo)

Una **Progressive Web App** es la web app actual con 3 mejoras:

1. **Manifest:** permite que el usuario la agregue al home screen como si fuera una app.
2. **Service Worker:** caché básico para que cargue más rápido.
3. **Push notifications** (opcional, requiere permiso): recordatorio de registrar comidas.

El usuario instala NutriApp desde el browser con "Agregar a pantalla de inicio" → aparece el ícono en el launcher, se abre sin barra del browser, parece una app nativa. **Funciona en Android y iOS.** Costo: mínimo, sin pasar por tiendas.

Lo que hay que agregar:

```
apps/web/public/manifest.json → nombre, íconos, colores
apps/web/public/sw.js         → service worker básico
<link rel="manifest"> en el layout
```

Fase 3 — App nativa (Solo si hay tracción real)

Si con 500+ usuarios activos el 60%+ viene desde mobile y la retención en web se estanca, ahí tiene sentido construir con **React Native + Expo**. Reutilizás lógica de negocio y tipado TypeScript, pero escribís componentes nativos. El backend de Express se queda igual.

No empezar acá. Construir una app nativa sin validar el producto es tirar 3-6 meses en infraestructura sin saber si hay demanda.

Veredicto

```
Hoy:           Web App (ya lista) → lanzar
Semana 2:      PWA (agregar manifest + service worker) → parece app nativa
Mes 3+:        Evaluar con datos reales si vale la pena la app nativa
```

La web app + PWA cubre el 80% de las ventajas de una app nativa, evita el 30% de Apple, no requiere aprobación de tiendas y permite lanzar hoy. La app nativa es una decisión futura basada en datos, no una necesidad del MVP.